

中山大學

SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab11-CUDA 卷积计算

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2026 年 6 月 15 日

1 实验目的

- 理解神经网络中二维卷积的基本计算过程，以及 stride 和 padding 对输出尺寸与计算量的影响。
- 掌握 CUDA 直接卷积（滑窗法）的实现方法，理解线程块配置、任务映射和访存方式对性能的影响。
- 掌握 im2col 方法，将卷积转换为矩阵乘法并使用已有的 CUDA 矩阵乘法程序完成计算。
- 学习使用 cuDNN 卷积接口，对比直接卷积、im2col 方法和 cuDNN 的运行性能。

2 实验内容

本实验实现神经网络中的二维卷积操作。卷积核不进行翻转（即互相关操作），不考虑偏置项，bias 设置为 0。为保证不同实现之间能够公平比较，统一采用以下实验设置：

- 输入为 $N \times N \times 3$ 的单精度数据（NCHW 布局），batch size 为 1；
- 使用一个 $3 \times 3 \times 3$ 卷积核，输出通道数为 1；
- padding 设置为 1，stride 分别设置为 1、2 和 3；
- 选择了 256×256、512×512、1024×1024 三组输入规模作为主要对比组；三种实现采用相同的输入和卷积参数。

输出尺寸计算公式为：

$$N_{\text{out}} = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1.$$

其中，P、K 和 S 分别表示 padding、卷积核大小和 stride。

2.1 CUDA 直接卷积

使用 CUDA 实现直接卷积（滑窗法），由 CUDA 线程计算各个输出位置的卷积结果。每个线程负责输出特征图上的一个像素点，遍历 3 个输入通道和 3×3 卷积核的 9 个位置进行乘加运算，边界超出范围的值视为 0（零填充）。测试不同输入图像大小、stride 和线程块大小下的运行时间；自行设计并说明输出任务到 CUDA 线程的映射方式，至少比较两种不同的任务粒度或线程映射方式。这里比较的是线程任务映射方式，而不是不同的卷积算法。

直接卷积核函数实现如下：

```
__global__ void conv2d_direct(float *in, float *ker, float *out,
                              int N, int C, int K, int P, int S, int
                                 N_out) {
    int ow = blockIdx.x * blockDim.x + threadIdx.x;
    int oh = blockIdx.y * blockDim.y + threadIdx.y;
```

```
    if (oh >= N_out || ow >= N_out) return;
    float sum = 0.0f;
    for (int c = 0; c < C; c++)
        for (int kh = 0; kh < K; kh++)
            for (int kw = 0; kw < K; kw++) {
                int ih = oh * S + kh - P, iw = ow * S + kw - P;
                if ((unsigned)ih < (unsigned)N && (unsigned)iw <
                    (unsigned)N)
                    sum += in[c * N * N + ih * N + iw]
                         * ker[c * K * K + kh * K + kw];
            }
    out[oh * N_out + ow] = sum;
}
```

Listing 1: 直接卷积核函数

2.2 im2col 方法

使用 im2col 将各个滑动窗口展开为矩阵，将卷积核展开为向量，再使用矩阵-向量乘法完成卷积计算。im2col 将输入张量 $[C][H][W]$ 中每个滑动窗口（$C \times K \times K = 27$ 个元素）展平为列向量，所有窗口组合成矩阵 col_matrix，维度为 $M_{patches} \times K_{dim}$，其中 $M_{patches} = N_{out} \times N_{out}$ 为输出像素总数，$K_{dim} = C \times K \times K = 27$ 为每个窗口展平后的维度。矩阵乘法采用 CUDA 实现的矩阵-向量乘法核函数，每个线程计算一个输出元素（col_matrix 的一行）与卷积核向量的内积。

im2col 展开核函数：

```
__global__ void im2col(float *in, float *col,
                       int N, int C, int K, int P, int S, int N_out) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int M = N_out * N_out;
    if (idx >= M) return;
    int oh = idx / N_out, ow = idx % N_out;
    int K_dim = C * K * K, base = idx * K_dim;
    for (int c = 0; c < C; c++)
        for (int kh = 0; kh < K; kh++)
            for (int kw = 0; kw < K; kw++) {
                int ih = oh * S + kh - P, iw = ow * S + kw - P;
                float val = 0.0f;
                if ((unsigned)ih < (unsigned)N && (unsigned)iw <
                    (unsigned)N)
                    val = in[c * N * N + ih * N + iw];
                col[base + c * K * K + kh * K + kw] = val;
            }
}
```

Listing 2: im2col 核函数

矩阵-向量乘法核函数（GEMV）：

```
__global__ void gemv_simple(float *col, float *ker, float *out, int M, int
    K_dim) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= M) return;
    float sum = 0.0f;
    float *row = col + idx * K_dim;
    for (int i = 0; i < K_dim; i++) sum += row[i] * ker[i];
    out[idx] = sum;
}
```

Listing 3: 矩阵-向量乘法核函数

2.3 cuDNN 卷积

使用 cuDNN 9.1.0 提供的 Legacy API（基于描述符的 API）完成卷积操作。cuDNN 编程模型的核心流程为：首先创建 cuDNN 句柄作为所有操作的上下文；然后用描述符定义输入张量、卷积核、卷积操作和输出张量的属性（维度、数据类型、padding、stride 等）；接着调用 `cudnnFindConvolutionForwardAlgorithm` 自动基准测试并选择最优算法；获取该算法所需的 workspace 大小并分配 GPU 内存；最后调用 `cudnnConvolutionForward` 执行卷积。使用完毕后释放描述符、workspace 和句柄。

cuDNN 卷积的完整实现代码如下：

```
// 1. 创建cuDNN句柄
cudnnHandle_t handle;
cudnnCreate(&handle);

// 2. 描述输入张量 [N=1, C=3, H=N, W=N], NCHW布局
cudnnTensorDescriptor_t in_desc;
cudnnCreateTensorDescriptor(&in_desc);
cudnnSetTensor4dDescriptor(in_desc, CUDNN_TENSOR_NCHW, CUDNN_DATA_FLOAT,
                           1, C, N, N);  // batch=1, channels=3, H, W

// 3. 描述卷积核 [K_out=1, C=3, K_h=3, K_w=3]
cudnnFilterDescriptor_t flt_desc;
cudnnCreateFilterDescriptor(&flt_desc);
cudnnSetFilter4dDescriptor(flt_desc, CUDNN_DATA_FLOAT, CUDNN_TENSOR_NCHW,
                           K_out, C, K, K);

// 4. 描述卷积操作: padding=P, stride=S, dilation=1, cross-correlation
```

```
cudnnConvolutionDescriptor_t conv_desc;
cudnnCreateConvolutionDescriptor(&conv_desc);
cudnnSetConvolution2dDescriptor(conv_desc, P, P, S, S, 1, 1,
                                CUDNN_CROSS_CORRELATION,
                                   CUDNN_DATA_FLOAT);

// 5. 描述输出张量 [N=1, K_out=1, H=N_out, W=N_out]
cudnnTensorDescriptor_t out_desc;
cudnnCreateTensorDescriptor(&out_desc);
cudnnSetTensor4dDescriptor(out_desc, CUDNN_TENSOR_NCHW, CUDNN_DATA_FLOAT,
                           1, K_out, N_out, N_out);

// 6. 自动选择最优前向卷积算法（cuDNN内部对多种算法做基准测试）
int req_cnt = 8, ret_cnt = 0;
cudnnConvolutionFwdAlgoPerf_t perf[8];
cudnnFindConvolutionForwardAlgorithm(handle,
    in_desc, flt_desc, conv_desc, out_desc, req_cnt, &ret_cnt, perf);
cudnnConvolutionFwdAlgo_t algo = perf[0].algo;  // 取最快的算法

// 7. 获取该算法所需的workspace大小并分配
size_t ws_size = 0;
cudnnGetConvolutionForwardWorkspaceSize(handle,
    in_desc, flt_desc, conv_desc, out_desc, algo, &ws_size);
void *d_workspace = NULL;
if (ws_size > 0) cudaMalloc(&d_workspace, ws_size);

// 8. 执行卷积（计时仅包含此调用）
float alpha = 1.0f, beta = 0.0f;
cudnnConvolutionForward(handle,
    &alpha, in_desc, d_input,
    flt_desc, d_kernel,
    conv_desc, algo,
    d_workspace, ws_size,
    &beta, out_desc, d_output);

// 9. 释放资源
cudnnDestroyTensorDescriptor(in_desc);
cudnnDestroyTensorDescriptor(out_desc);
cudnnDestroyFilterDescriptor(flt_desc);
cudnnDestroyConvolutionDescriptor(conv_desc);
if (d_workspace) cudaFree(d_workspace);
cudnnDestroy(handle);
```

Listing 4: cuDNN 卷积实现

cuDNN 通过描述符（Descriptor）抽象出张量形状、内存布局、卷积参数等信息，使得

同一套 API 可以适配不同的输入尺寸和卷积配置。`cudnnFindConvolutionForwardAlgorithm` 内部对可用算法（IMPLICIT_GEMM、GEMM、Winograd、FFT 等）逐一进行快速基准测试，返回性能排序后的算法列表，程序取最快的算法使用。与前两种方法一样，只统计 GPU 计算阶段时间（仅统计 `cudnnConvolutionForward` 的执行时间），预热 10 次后测量 50 次取平均值。

3 实验结果

本节记录所有性能数据。所有 kernel 时间使用 CUDA Event 计时（`cudaEventRecord`），每组实验预热 10 次后测量 50 次取平均值，仅统计 GPU 计算阶段时间，不包含显存申请、程序初始化和主机与设备之间的数据传输时间。

关于时间尺度的说明：本实验所有时间数据均通过 CUDA Event 计时获取，每组配置预热 10 次后测量 50 次取平均值，计时结果稳定（多次运行波动 $< 5\%$）。由于只统计 GPU kernel 执行阶段，不包含显存分配与 H2D/D2H 传输，且卷积规模最大仅约 10^6 输出像素，因此时间处于微秒量级。

以最大规模 1024×1024、stride=1 为例：输出像素约 1.05×10^6，总计算量约 2.83×10^7 次 FMA。若仅按 V100 单精度峰值（约 14 TFLOPS）估算，纯计算下限约 0.002 ms——**该数值仅供数量级参考，不代表实际可达性能**。卷积是访存密集型运算：无缓存优化时，每像素最多需从全局内存读取 27 个 float（约 108 bytes），10^6 像素对应约 108 MB 读取量，V100 的 HBM2 带宽约 900 GB/s，仅全局内存读取就需约 0.12 ms。实测 0.055 ms 处于计算下限与带宽下限之间，说明 GPU 的 L1/L2 缓存有效降低了实际全局内存访问量。对于 256×256 等更小规模，kernel launch 开销（约 3–5μs）在总时间中占比上升，但数据仍在合理范围。

3.1 线程块大小对直接卷积性能的影响

固定输入图像大小 $512 \times 512 \times 3$、stride=1 和任务映射方式（每线程 1 个输出像素，2D 线程块），测量不同线程块大小下 CUDA 直接卷积的运行时间。

表 1: 不同线程块大小下 CUDA 直接卷积的运行时间（ms）

输入图像大小	线程块大小	运行时间
512×512	8×8	0.015599
	16×16	0.012436
	32×32	0.013580

结果分析：三种配置中，16×16（256 线程）性能最优。8×8（64 线程，仅 2 个 Warp）并行度不足，无法有效隐藏内存访问延迟，性能最差。32×32（1024 线程，已达 GPU 线程块上限）虽块内并行度高，但每个 SM 最多同时驻留 2 个此类线程块

（$2 \times 1024 = 2048$ 线程），线程块间切换的延迟隐藏能力较弱。16×16 每个 SM 可驻留 8 个线程块（$8 \times 256 = 2048$ 线程，64 个 Warp），在保持高占用率的同时确保足够的线程块级并行度，实现了并行度与资源利用率的最佳平衡。

3.2 任务划分与线程映射方式对性能的影响

固定输入图像大小 $512 \times 512 \times 3$、stride=1，保持约 256 线程/块，测量不同任务划分与线程映射方式下直接卷积的运行时间。

表 2: 不同任务划分与线程映射方式下直接卷积的运行时间（ms）

编号	任务划分与线程映射方式	运行时间
方式一	每线程 1 个输出像素，2D 线程块 16×16，Grid 覆盖 (W_{out}, H_{out})	0.012436
方式二	每线程 1 个输出像素，1D 线程块 256 线程，grid.y 映射输出行索引	0.012420
方式三（可选）	每线程 $2 \times 2 = 4$ 个输出像素（寄存器分块），16×16 线程块	0.041247

各方式说明：

方式一（2D 线程块映射）：将输出特征图的二维空间直接映射到 2D Grid 和 2D Block。每个线程索引 $(threadIdx.x, threadIdx.y)$ 对应输出空间的一个像素坐标 (ow, oh)。同一 Warp 内相邻 threadIdx.x 的线程访问空间相邻的输入数据，有利于 L2 缓存命中。

方式二（1D 行映射）：Grid 的 y 维度映射输出行索引，Block 包含 256 个 1D 线程处理该行的各列。每个线程块处理一行（或一行的一段），线程之间通过 blockIdx.y 区分不同行。此映射方式将输出行的概念显式化，在处理非正方形特征图时更灵活。

方式三（寄存器分块）：每线程计算 $2 \times 2 = 4$ 个相邻输出像素。将卷积核缓存到寄存器（27 个 float），4 个累加器也保留在寄存器中。目的是增加每线程计算密度以换取更少的线程调度开销。

结果分析：方式一和方式二性能几乎相同（差异 $< 0.3\%$），说明对于规整的 2D 卷积，两种线程组织方式在内存合并访存方面表现相似。方式三性能明显下降（约 3.3 倍）。从核函数结构分析，每线程需要维护 4 个输出累加器、27 个卷积核缓存值以及多个循环索引，寄存器需求远高于方式一；这会导致每个 SM 能同时驻留的线程数减少，降低延迟隐藏能力（可通过 `nvcc --ptxas-options=-v` 进一步确认实际寄存器用量）。此外 2×2 分块引入了更多内层分支判断（需逐一检查 4 个输出位置是否越界），也增加了指令开销。分块策略在更大卷积核（如 5×5 以上）或更深通道数场景中可能更有优势。

3.3 不同卷积实现的性能对比

在相同输入图像大小、stride、padding 和卷积核设置下，测量直接卷积、im2col 方法和 cuDNN 的运行时间。im2col 总时间 = im2col 数据展开时间 + GEMV 矩阵乘法时间。cuDNN 使用 `cudnnFindConvolutionForwardAlgorithm` 自动选择算法。根据实验观察，cuDNN 在本实验配置下未表现出 Winograd 算法常见的性能优势，推测其可能选择了基于 GEMM 的实现路径（cuDNN 9.x 在 batch=1、FP32 条件下，启发式算法可能判定 Winograd 的变换开销超过了计算收益）。

表 3: 不同卷积实现的性能对比（时间单位：ms）

输入图像大小	stride	直接卷积	im2col 方法	cuDNN
256×256	1	0.006204	0.103764	0.024509
	2	0.004524	0.024716	0.016612
	3	0.004694	0.015410	0.016538
512×512	1	0.012459	0.977715	0.023800
	2	0.005668	0.114914	0.012028
	3	0.005032	0.037411	0.012154
1024×1024	1	0.055440	4.700619	0.084531
	2	0.021998	0.970368	0.031362
	3	0.018836	0.355535	0.023099

4 实验分析

4.1 问题一

分析输入图像大小、stride 和线程块大小对直接卷积运行时间的影响；说明哪种线程块配置性能较好，并结合并行度和硬件资源使用分析原因。

回答：

（1）输入图像大小的影响：直接卷积运行时间随输入尺寸增大而增加。以 stride=1 为例，当输入从 256×256 增至 1024×1024 时，输出像素数从 $256^2 = 65,536$ 增至 $1024^2 = 1,048,576$（增长 16 倍），而运行时间从 0.006204 ms 增至 0.055440 ms（增长约 9 倍）。时间增长低于计算量的线性增长，体现了 GPU 在大规模并行计算中的优势——更大的问题规模能更充分地利用 GPU 的多个 SM（流式多处理器）和数千个 CUDA 核心，有效摊薄了 kernel 启动开销。

（2）stride 的影响：stride 增大导致输出尺寸减小，运行时间显著下降。以 512×512 输入为例，stride=1 时输出为 512×512（262,144 像素），stride=3 时输出为 171×171（29,241 像素），计算量减少约 89%，运行时间从 0.012459 ms 降至 0.005032 ms（减少约 60%）。注意 stride 不影响每个输出像素的计算量（始终为 $C \times K^2 = 27$ 次乘加运算），

仅改变总输出像素数。运行时间的减少比例低于计算量的减少比例，原因是 kernel 启动开销（约 3–5 μs）在总时间中占比随计算量减少而增大。

（3）线程块大小的影响：16×16（256 线程 =8 个 Warp）表现最优，是推荐配置。理论分析如下：以本实验使用的 GPU（计算能力 7.0）为例，每个 SM 最多驻留 2048 个线程（64 个 Warp）或 32 个线程块。8×8（64 线程 =2 Warp/块）需 32 个线程块方能占满 SM（恰达上限），但频繁的线程块调度带来开销；32×32（1024 线程 =32 Warp/块）仅能驻留 2 个线程块（$2 \times 1024 = 2048$ 线程），块间切换次数少导致延迟隐藏不足；16×16（256 线程 =8 Warp/块）可驻留 8 个线程块（$8 \times 256 = 2048$ 线程），在 Warp 调度灵活性和线程块级并行度之间取得最佳平衡。

4.2 问题二

说明所采用的任务划分与线程映射方式，分析不同方式对任务粒度、负载均衡和全局内存访问的影响。

回答：

方式一（2D 线程块映射）将输出空间 (H_{out}, W_{out}) 直接映射到 2D Grid，线程索引与输出坐标一一对应，代码简洁直观。在全局内存访问方面，同一 Warp 内相邻 threadIdx.x 的 32 个线程访问输入数据时具有空间局部性——它们对应的 ih, iw 坐标相差不大，有利于 L2 缓存行命中。任务粒度方面，每线程 27 次乘加运算，属于细粒度任务划分，线程总数多（对 512×512 为 262,144 个线程），线程调度开销相对较大但被大量计算充分隐藏。

方式二（1D 行映射）将输出行显式化为 grid.y 维度，block 含 256 个 1D 线程处理行内各列。此方式与方式一的本质区别在于线程块内线程的空间关系：方式一中同一 block 的线程覆盖 16×16 的 2D 区域，方式二中同一 block 的线程覆盖一行中 256 个连续像素。从合并访存角度，两种方式的同一 Warp 内相邻线程访问空间相邻的输入数据，访存效率相当。方式二的优点是对非正方形特征图（如 $H_{out} \gg W_{out}$）更灵活，可通过调整 grid.y 和 block.x 实现更均衡的负载分配。

方式三（寄存器分块）每线程处理 $2 \times 2 = 4$ 个输出像素。任务粒度从细粒度（1 像素/线程）变为中粒度（4 像素/线程），总线程数减少至原来的 1/4。理想情况下，通过减少线程调度开销和增加数据重用（相邻 2×2 像素的输入窗口高度重叠），应能获得性能提升。但实验结果显示性能反而下降（约为方式一的 3.3 倍）。从核函数结构看，每线程需维护 4 个累加器和 27 个卷积核寄存器值，加上循环索引等变量，寄存器需求远高于方式一。GPU 每个 SM 的寄存器总量是固定的，单线程寄存器用量越大，SM 能同时驻留的线程数就越少，延迟隐藏能力随之下降。寄存器压力增大导致的占用率降低，是分块策略在当前场景性能不佳的主要原因（可通过 `nvcc --ptxas-options=-v` 查看编译器报告的寄存器用量来进一步验证这一推测）。

三种方式的负载均衡均表现良好——每个线程的计算量完全相同（方式一/二为 27 次乘加，方式三为 108 次乘加），不存在负载不均导致的线程等待问题。

4.3 问题三

对比直接卷积与 im2col 方法的性能，分析 im2col 的数据展开与额外内存访问开销，以及规则矩阵乘法可能带来的收益。

回答：

实验结果表明，对于 $3\times3\times3$ 小卷积核，直接卷积在所有配置下都快于 im2col 方法。以 512×512、stride=1 为例：直接卷积需要 0.012459 ms，im2col 方法总计 0.977715 ms（im2col 展开 0.900019 ms + GEMV 0.077696 ms），直接卷积快约 78 倍。以 1024×1024、stride=1 为例差距更大：直接卷积 0.055440 ms vs im2col 4.700619 ms（约 85 倍）。

im2col 性能低于直接卷积的核心原因：

(a) 内存数据量膨胀：im2col 将 $N\times N\times3$ 的输入（$3N^2$ 个 float）展开为 $N_{out}^2\times27$ 的矩阵。当 $N=512$、stride=1 时，输入为 $512^2\times3=786,432$ 个 float（约 3 MB），im2col 矩阵为 $512^2\times27=7,077,888$ 个 float（约 27 MB）——数据量膨胀了 9 倍（恰好等于 K^2）。大量冗余的内存写入成为瓶颈。

(b) im2col 展开开销：im2col 核函数对每个输出位置需执行 27 次全局内存写入（展开矩阵的 27 个元素），加上边界检查和索引计算。而直接卷积核函数每个线程仅需 1 次全局内存写入（输出结果），所有中间累加在寄存器中完成。对于 512×512 输出，im2col 需要 $512^2\times27\approx7.08\times10^6$ 次全局内存写入，直接卷积仅需 $512^2\approx2.62\times10^5$ 次写入（差距 27 倍）。

(c) GEMV 阶段的访存开销：矩阵-向量乘法需读取整个 col_matrix（27 MB），加上对卷积核的重复读取（虽然只有 27 个 float，但被每个线程读取一次）。而直接卷积的输入数据读取在计算发生时进行，配合 L1/L2 缓存可显著减少实际的内存带宽消耗。

im2col 的适用场景：当卷积核较大（如 5×5、7×7 或更大）时，直接卷积的嵌套循环（$C\times K\times K$ 次迭代）难以被编译器有效展开优化，分支预测失败率升高。而 im2col 将问题统一转换为矩阵乘法后，可利用高度优化的 cuBLAS SGEMM/SGEMV 实现（利用 Tensor Core、共享内存分块等硬件特性）。此外，当 batch size 较大或输出通道数较多时，im2col 产生的矩阵维度更大，GEMM 的计算密度（FLOP/byte）随之提高，im2col 方法的相对劣势会逐渐减小甚至反转。

4.4 问题四

对比自行实现的卷积与 cuDNN 的性能；如果性能低于 cuDNN，结合数据复用、合并访存、共享内存、常量内存和线程块配置说明可能的改进方向。

回答：

实测 cuDNN 9.1.0 的结果已列于表 3。将三种方法对比分析：

（1）直接卷积 vs cuDNN：

从实测数据看：在 $3\times3\times3$、batch=1 的配置下，直接卷积在多数测试中的耗时低于 cuDNN。以 512×512 stride=1 为例：直接卷积 0.012459 ms vs cuDNN 0.023800 ms

（直接卷积约为 cuDNN 耗时的 52%）。256×256 stride=1 时：直接卷积 0.006204 ms vs cuDNN 0.024509 ms（直接卷积约为 cuDNN 耗时的 25%）。

这一反直觉结果的原因：

- **cuDNN 未选择 Winograd 路径**：从 cuDNN 返回的算法名称来看，本实验配置下 cuDNN 倾向于选择基于 GEMM 的实现而非 Winograd。GEMM 路径内部需要 im2col 式的数据重排，对于 batch=1 的极薄矩阵，计算密度低，数据重排开销占主导；
- **Winograd 未被触发**：cuDNN 9.x 中 Winograd 算法可能需要特定的 tile 配置、对齐条件或 FP16 精度才会被选中。在 FP32、batch=1 下，启发式算法判定 Winograd 的变换开销超过了计算节省量；
- **单 batch 下的固定开销**：cuDNN 面向大 batch 深度学习训练优化。batch=1 时，算法 auto-tune、workspace 分配和描述符管理的固定开销占总执行时间比例较高；
- **直接卷积无数据重排开销**：3×3 小卷积核下直接滑窗法不需要 im2col 式的数据变换，每线程仅 27 次乘加，无额外内存开销。

但 1024×1024 stride=1 时差距缩小（直接 0.055440 vs cuDNN 0.084531 ms），趋势上大计算量下 cuDNN 的优化优势在逐步显现。

（2）cuDNN vs im2col：cuDNN 在所有配置下均明显快于 im2col（1024×1024 stride=1: 0.084531 vs 4.700619 ms，im2col 耗时约为 cuDNN 的 55 倍），说明即使 cuDNN 未启用 Winograd，其内部高度优化的实现也比简单的手工 GEMV 高效得多。

（3）改进方向：虽然本实验在 batch=1、小卷积核场景下直接卷积在多数配置中略优于 cuDNN，但这种优势主要来源于问题规模较小以及 cuDNN 固定开销（算法选择、描述符管理）占比较高。在更大卷积核、多输出通道或更大 batch 等更典型的深度学习场景下，cuDNN 通常具有明显优势。因此以下优化方向仍有实际意义：

(a) 共享内存输入分块：将输入 tile 加载到共享内存。3×3 卷积和 16×16 线程块需约 $(16+2)^2 \times 3 = 972$ 个 float（约 3.8 KB），每个输入像素被 9 个线程共享，全局内存读取减至 1/9。

(b) 常量内存存储卷积核：将 27 个 float 的卷积核存入常量内存（`__constant__`）。同一 Warp 内广播访问，消除每线程的卷积核全局内存读取。

(c) 向量化访存：使用 `float4` 一次读取 4 个 float，将内存事务数减至 1/4，更充分利用 GPU 的高带宽显存（如本实验 GPU 的 900 GB/s HBM2 带宽）。

(d) Winograd 算法：对 3×3 卷积实现 $F(2 \times 2, 3 \times 3)$ Winograd，可将乘法次数降低约 2.25 倍。在特定的小卷积核场景下，Winograd 算法有望进一步减少计算量，从而获得一定性能提升。

综合而言，当前直接卷积在 batch=1、小卷积核这一特定场景下耗时低于 cuDNN，主要得益于问题规模小、无额外数据重排开销。引入共享内存和常量内存优化后可进一步降低访存开销；在更大 batch、更大卷积核等更通用的场景下，共享内存分块和 Winograd 算法是进一步缩小与 cuDNN 差距的关键方向。